Table of Contents

1. Relational Calculus (Tuple & Domain) 3

2. Dependency Preservation 5

3. Query Optimization Algorithms 6

4. Database Security 8

5. Access Control Models: DAC, MAC and RBAC 9

6. Insertion and Deletion 11

7. SQL Injection 13

8. Data Warehousing and Data Mining 15

Conclusion 17

1. Relational Calculus (Tuple & Domain)

Relational calculus is a *non-procedural* (declarative) query language. Unlike relational algebra, which tells the system *how* to get the result step by step, relational calculus only describes *what* result is wanted. It is based on first-order predicate logic. It comes in two flavours: **Tuple Relational Calculus (TRC)** and **Domain Relational Calculus (DRC)**.

1.1 Tuple Relational Calculus (TRC)

In TRC, variables range over **tuples** (rows) of a relation. A query has the general form:

```
{ t | P(t) }
```

which reads: *"the set of all tuples t such that predicate P is true for t."* Quantifiers used: **∃** (there exists) and **∀** (for all). Logical connectives: ∧ (AND), ∨ (OR), ¬ (NOT), ⇒ (implies).

Example schema: Student(sid, name, age, dept) | Enrolled(sid, cid, grade) | Course(cid, title)

Query	TRC expression
All students older than 20	{ t | Student(t) ∧ t.age > 20 }
Names of CS students	{ t.name | Student(t) ∧ t.dept = 'CS' }
Names of students enrolled in course C101	{ t.name | Student(t) ∧ ∃e (Enrolled(e) ∧ e.sid = t.sid ∧ e.cid = 'C101') }
Students enrolled in *all* courses	{ t | Student(t) ∧ ∀c (Course(c) ⇒ ∃e (Enrolled(e) ∧ e.sid = t.sid ∧ e.cid = c.cid)) }

1.2 Domain Relational Calculus (DRC)

In DRC, variables range over **domain values** (attribute values) rather than whole tuples. The general form is:

```
{ <x1, x2, ..., xn> | P(x1, x2, ..., xn) }
```

Query	DRC expression
Names and ages of students older than 20	{ | ∃s, d (Student(s, n, a, d) ∧ a > 20) }
Names of CS students	{ | ∃s, a (Student(s, n, a, 'CS')) }
Names of students enrolled in C101	{ | ∃s, a, d (Student(s, n, a, d) ∧ Enrolled(s, 'C101', g)) }

1.3 TRC vs DRC

Feature	Tuple Relational Calculus	Domain Relational Calculus
Variable ranges over	Whole tuples (rows)	Individual attribute values
Notation	{ t | P(t) }	{ <x1,...,xn> | P(x1,...,xn) }
Attribute access	Dot notation: t.name	Positional: Student(s, n, a, d)
Verbosity	More compact	Longer (one variable per attribute)

Feature	Tuple Relational Calculus	Domain Relational Calculus
Basis for	SQL	QBE (Query-By-Example)

1.4 Safe Expressions

An expression is **safe** if it guarantees a finite result. For example, { t | ¬Student(t) } is unsafe because infinitely many tuples are not in Student. Safe TRC/DRC is restricted so every output value comes from the database; safe calculus has the **same expressive power as relational algebra** (Codd's Theorem).

2. Dependency Preservation

2.1 Background: Functional Dependencies

A **functional dependency (FD)** X → Y means that if two tuples agree on attributes X, they must agree on attributes Y. When a relation R is decomposed into smaller relations (for normalization), we want the decomposition to have two key properties: **lossless join** and **dependency preservation**.

2.2 Definition

A decomposition of R into R_1, R_2, ..., R_n is **dependency preserving** if every functional dependency in F (the original set) can be enforced by checking the FDs on the individual decomposed relations alone — without joining them back together.

Formally, let F_i be the projection of F onto R_i (FDs of F^+ whose attributes are all in R_i). The decomposition is dependency preserving if:

```
(F1 ∪ F2 ∪ ... ∪ Fn)+  =  F+
```

2.3 Why It Matters

- Without it, checking a constraint requires an expensive **join** of the decomposed tables on every insert/update.
- It lets the DBMS enforce integrity cheaply at the single-table level.
- It is a goal of **3NF synthesis**: 3NF decomposition is always lossless *and* dependency preserving, whereas BCNF decomposition is always lossless but *may not* preserve dependencies.

2.4 Worked Example

Let R(A, B, C) with F = { A → B, B → C }. Decompose into R_1(A, B) and R_2(B, C).

- F_1 (on R_1) contains A → B.
- F_2 (on R_2) contains B → C.
- $F_1 \cup F_2$ = { A → B, B → C } = F, so **all FDs are preserved**. ✔

Counter-example: R(A, B, C) with F = { AB → C, C → B }. Decomposing into R_1(A, C) and R_2(C, B) gives F_1 = {} and F_2 = { C → B }. The dependency AB → C cannot be checked in either table and cannot be derived, so it is **not preserved**. ✘

2.5 Algorithm to Test Preservation

```
For each FD  X -> Y  in F:
    result = X
    repeat until result does not change:
        for each Ri in the decomposition:
            t = (result ∩ Ri)+  ∩ Ri        # closure computed using F
            result = result ∪ t
    if Y ⊆ result: the FD is preserved
    else:          the FD is NOT preserved
If every FD in F is preserved, the decomposition is dependency preserving.
```

3. Query Optimization Algorithms

Query optimization is the process by which the DBMS chooses the most efficient execution plan for a query. The user writes *what* they want in SQL; the optimizer decides *how* to execute it, aiming to minimise cost (disk I/O, CPU time, memory, network).

3.1 Phases of Query Processing

- **Parsing & translation** – SQL is checked for syntax/semantics and converted to a relational algebra expression (parse tree).
- **Optimization** – equivalent expressions/plans are generated and the cheapest is chosen.
- **Evaluation** – the execution engine runs the chosen plan and returns results.

3.2 Heuristic (Rule-Based) Optimization

Uses general rules that almost always improve performance, applied on the query tree:

- **Perform selections (σ) as early as possible** – reduces the number of tuples early.
- **Perform projections (π) early** – removes unneeded columns.
- **Combine selection with Cartesian product to form joins** (σ + × → ⋈).
- **Break up conjunctive selections**: $\sigma_{c1 \wedge c2}(R) = \sigma_{c1}(\sigma_{c2}(R))$.
- **Use associativity/commutativity of joins** to start with the most restrictive (smallest) join.
- **Pipeline operations** instead of materialising intermediate results.

3.3 Heuristic Optimization Algorithm (Steps)

```
1. Convert the query into an initial query tree (canonical form).
2. Push SELECT operations down the tree (as close to the leaves as possible).
3. Rearrange leaf nodes so the most restrictive SELECTs execute first.
4. Replace CARTESIAN PRODUCT + SELECT with JOIN.
5. Push PROJECT operations down the tree.
6. Identify sub-trees that can be pipelined.
```

3.4 Cost-Based Optimization

The optimizer enumerates alternative plans, estimates the cost of each using **statistics from the system catalog** (number of tuples, number of blocks, number of distinct values, index height, selectivity), and picks the plan with the lowest estimated cost.

Operation	Typical algorithms	Cost idea (blocks)
Selection	Linear scan, binary search, B+-tree index scan, hash index	Scan: b_r; Index: height + matching blocks
Join	Nested-loop, Block nested-loop, Index nested-loop, Sort-merge, Hash join	BNL: $b_r + b_r \times b_s$ ($M - 2$ buffers reduce further)
Sorting	External merge sort	$\approx 2 \times b \times (\lceil \log_{M-1}(b/M) \rceil + 1)$
Aggregation / Duplicate removal	Sorting or hashing	Depends on sort/hash cost

3.5 Join Order: Dynamic Programming (System R Style)

For a query joining n relations there are many join orders. System R's optimizer uses **dynamic programming** to find the best order, building the best plan for subsets of relations of increasing size and keeping only the cheapest plan per subset (and per “interesting order”).

```
for each relation R:                    bestPlan({R}) = cheapest access path (scan / index)
for size = 2 to n:
    for each subset S of relations with |S| = size:
        bestPlan(S) = cheapest over all  (S' , Ri)  where S = S' ∪ {Ri}
                      of  join( bestPlan(S'), Ri )
return bestPlan({R1, ..., Rn})
```

Restricting to **left-deep** join trees reduces the search space and allows pipelining. For larger queries (typically > 10–15 tables) **greedy**, **genetic** or **randomised** algorithms are used instead.

3.6 Example

```
SELECT s.name FROM Student s, Enrolled e
WHERE s.sid = e.sid AND e.cid = 'C101';

Naive plan :  π name ( σ s.sid=e.sid ∧ cid='C101' ( Student × Enrolled ) )
Optimized  :  π name ( Student ⋈ sid  ( σ cid='C101' (Enrolled) ) )
              -> selection pushed down first, product replaced by join
```

4. Database Security

Database security refers to the collection of measures used to protect a database against threats to its **confidentiality, integrity, and availability** (the CIA triad).

4.1 Security Goals

Goal	Meaning	Example threat
Confidentiality	Only authorised users can read data	Data leak, unauthorised access
Integrity	Data is accurate and modified only by authorised users	Tampering, unauthorised updates
Availability	Data is accessible when needed	DoS attack, ransomware

4.2 Common Threats

- SQL injection and other application-level attacks
- Privilege abuse / insider threats
- Weak authentication, default or stolen passwords
- Unpatched software vulnerabilities
- Backup theft and physical media loss
- Denial-of-service attacks
- Inference attacks on statistical databases

4.3 Security Mechanisms

- **Authentication** – verifying identity (passwords, multi-factor, certificates).
- **Authorization / Access control** – deciding what each user may do (DAC, MAC, RBAC – see Section 5).
- **Encryption** – data at rest (TDE, column encryption) and in transit (TLS/SSL).
- **Auditing & logging** – recording who accessed or changed what, and when.
- **Views** – expose only a subset of data to certain users.
- **Backup and recovery** – protect against loss and ransomware.
- **Firewalls, network segmentation, and patching**.
- **Data masking / anonymisation** for test environments.
- **Least privilege principle** – grant only the minimum permissions required.

4.4 SQL Authorization Example

```
GRANT SELECT, UPDATE ON Student TO clerk;
REVOKE UPDATE ON Student FROM clerk;
CREATE VIEW cs_students AS SELECT sid, name FROM Student WHERE dept = 'CS';
GRANT SELECT ON cs_students TO intern;
```

5. Access Control Models: DAC, MAC and RBAC

Access control models define *who* (subject) can perform *which* operations (read, write, execute) on *which* resources (objects).

5.1 Discretionary Access Control (DAC)

The **owner** of an object decides who else may access it and can pass those privileges to others (e.g., SQL `GRANT ... WITH GRANT OPTION`). Access rights are often represented by an **access control matrix** or access control lists (ACLs).

- **Pros:** flexible, easy to implement, widely used (SQL, UNIX file permissions).
- **Cons:** vulnerable to **Trojan horse** attacks and accidental over-sharing since users control permissions; hard to enforce an organisation-wide policy.

```
GRANT SELECT ON Employee TO alice WITH GRANT OPTION;   -- alice may re-grant SELECT
```

5.2 Mandatory Access Control (MAC)

Access is controlled by a **central authority** using **security labels**. Each subject has a *clearance level* and each object has a *classification level* (e.g., Top Secret > Secret > Confidential > Unclassified). Users cannot change these labels. MAC is used in military and government systems. It is typically enforced by the **Bell–LaPadula model**:

- **Simple Security Property (no read up):** a subject may read an object only if clearance ≥ classification.
- **Star (*) Property (no write down):** a subject may write to an object only if classification ≥ the subject's clearance — preventing leakage of secrets to lower levels.
- **Pros:** very strong protection; resists Trojan horses and insider leaks.
- **Cons:** rigid, complex to administer, less flexible for commercial use.

5.3 Role-Based Access Control (RBAC)

Permissions are assigned to **roles** (e.g., Manager, Teller, Auditor), and users are assigned to roles. A user acquires permissions only through the roles they hold. This mirrors organisational structure.

```
CREATE ROLE hr_manager;
GRANT SELECT, INSERT, UPDATE ON Employee TO hr_manager;
GRANT hr_manager TO alice;      -- alice now has all of the role's privileges
```

- **Pros:** simple administration, supports **least privilege**, separation of duties, role hierarchies (e.g., Manager inherits Employee).
- **Cons:** role explosion in large organisations; initial role design requires effort.

5.4 Comparison

Aspect	DAC	MAC	RBAC
Who controls access	Object owner	Central security administrator	Administrator via roles

Aspect	DAC	MAC	RBAC
Basis of decision	User identity & granted privileges	Security labels (clearance vs classification)	User's role
Flexibility	High	Low	Medium–High
Security strength	Lowest	Highest	Medium–High
Administration	Decentralised, hard to audit	Centralised, strict	Centralised, scalable
Typical use	Commercial DBMS, file systems	Military, government	Enterprises, banks, hospitals

6. Insertion and Deletion

Insertion and deletion are core **data manipulation (DML)** operations. In relational theory they must preserve integrity constraints: **domain, key, entity integrity, and referential integrity**.

6.1 Insertion

SQL INSERT adds one or more new tuples to a relation.

```
INSERT INTO Student (sid, name, age, dept) VALUES (101, 'Riya', 20, 'CS');
INSERT INTO Student VALUES (102, 'Arjun', 21, 'IT');
INSERT INTO Alumni SELECT * FROM Student WHERE age > 24;   -- insert from a query
```

Constraints checked on insertion:

- **Domain constraint** – value must be of the correct type/range.
- **Key constraint** – primary key value must be unique.
- **Entity integrity** – primary key must not be NULL.
- **Referential integrity** – a foreign key value must exist in the referenced table (or be NULL).

6.2 Deletion

SQL DELETE removes tuples satisfying a condition. Without a WHERE clause, all rows are deleted.

```
DELETE FROM Student WHERE sid = 101;
DELETE FROM Student WHERE dept = 'IT' AND age > 25;
DELETE FROM Student;          -- deletes ALL rows (table structure remains)
```

Constraint on deletion: deleting a referenced tuple may violate referential integrity. Options defined with the foreign key:

Action	Effect
RESTRICT / NO ACTION	Reject the deletion if referencing tuples exist
CASCADE	Automatically delete the referencing tuples too
SET NULL	Set the foreign key in referencing tuples to NULL
SET DEFAULT	Set the foreign key to its default value

```
CREATE TABLE Enrolled (
    sid INT, cid VARCHAR(10),
    FOREIGN KEY (sid) REFERENCES Student(sid) ON DELETE CASCADE
);
```

6.3 DELETE vs TRUNCATE vs DROP

Command	Type	Removes	Rollback?	WHERE allowed?
DELETE	DML	Selected rows	Yes	Yes

Command	Type	Removes	Rollback?	WHERE allowed?
TRUNCATE	DDL	All rows (keeps structure)	Usually no (DBMS-dependent)	No
DROP	DDL	Entire table (data + structure)	No	No

6.4 Insertion & Deletion Anomalies

In poorly designed (unnormalised) tables, these operations cause **anomalies**:

- **Insertion anomaly:** cannot add data about one entity without adding unrelated data. E.g., in a table Employee_Dept(emp_id, emp_name, dept_id, dept_name), a new department cannot be recorded until an employee is assigned to it.
- **Deletion anomaly:** deleting a tuple unintentionally loses other facts. E.g., deleting the last employee of a department also deletes the department's details.
- **Update anomaly:** the same fact repeated in many rows must be updated everywhere, else inconsistency results.

Solution: normalisation (1NF → 2NF → 3NF → BCNF) separates data so each fact is stored once.

7. SQL Injection

SQL injection (SQLi) is an attack where an adversary inserts malicious SQL code into an application's input so that it becomes part of a query executed by the database. It happens when user input is concatenated directly into SQL strings without proper handling. It is consistently listed among the top web-application vulnerabilities (OWASP Injection).

7.1 How It Works

Vulnerable code builds the query by string concatenation:

```
query = "SELECT * FROM users WHERE username = '" + user + "' AND password = '" + pwd + "'"
```

If the attacker enters `admin' --` as the username, the query becomes:

```
SELECT * FROM users WHERE username = 'admin' --' AND password = ''
-- everything after -- is a comment, so the password check is skipped
```

Another classic payload is `' OR '1'='1`, which makes the WHERE condition always true and returns every row.

7.2 Types of SQL Injection

Type	Description
In-band (Classic)	Attacker uses the same channel to inject and read results. Includes **error-based** and **UNION-based** injection.
Blind – Boolean	No data shown; attacker infers information from TRUE/FALSE differences in page behaviour.
Blind – Time-based	Attacker infers data from response delays (e.g., using sleep functions).
Out-of-band	Data is exfiltrated through another channel (DNS/HTTP requests).
Second-order (stored)	Malicious input is stored first and triggers when used later in another query.

7.3 Impact

- Authentication bypass and account takeover
- Reading sensitive data (passwords, personal and financial data)
- Modifying or deleting data (DROP TABLE, UPDATE)
- Privilege escalation and, in some configurations, remote command execution

7.4 Prevention

- **Parameterised queries / prepared statements** (primary defence) – the query structure is fixed; input is treated purely as data.
- **Stored procedures** (when written without dynamic SQL).
- **Input validation** using allow-lists (type, length, format).
- **Escaping** special characters (only as a secondary measure).
- **Least privilege** – the application's DB account should not have DROP/ADMIN rights.

- **Web Application Firewall (WAF)**, hide detailed error messages, regular security testing and patching.
- Use **ORMs** carefully (avoid raw string-built queries).

```
# Vulnerable                                        # Safe (parameterised)
cur.execute("SELECT * FROM users                    cur.execute(
   WHERE name='" + user + "'")                           "SELECT * FROM users WHERE name = %s", (user,))
```

8. Data Warehousing and Data Mining

8.1 Data Warehousing

A **data warehouse** is a large, centralised repository that integrates data from multiple heterogeneous sources and is designed for **analysis and decision support** rather than day-to-day transactions. According to W. H. Inmon, it is *"subject-oriented, integrated, time-variant, and non-volatile."*

- **Subject-oriented:** organised around business subjects (sales, customers, products).
- **Integrated:** data from different sources is cleaned and made consistent.
- **Time-variant:** stores historical data, enabling trend analysis.
- **Non-volatile:** data is loaded and read, not updated or deleted in routine operation.

8.2 Architecture and ETL

Data flows: **Operational sources (OLTP DBs, files, web) → ETL → Data warehouse / data marts → OLAP, reports, dashboards, data mining**.

- **Extract** – read data from source systems.
- **Transform** – clean, deduplicate, convert formats, apply business rules, aggregate.
- **Load** – write into the warehouse (initial and periodic incremental loads).

8.3 Dimensional Modelling

- **Fact table:** stores numeric measures (e.g., sales amount, quantity) with foreign keys to dimensions.
- **Dimension tables:** descriptive attributes (Time, Product, Customer, Location).
- **Star schema:** one central fact table directly linked to denormalised dimension tables.
- **Snowflake schema:** dimension tables are normalised into sub-dimensions.
- **Fact constellation (galaxy):** multiple fact tables sharing dimensions.

8.4 OLAP Operations

Operation	Description
Roll-up	Aggregate to a higher level (city → country)
Drill-down	Move to more detailed data (year → quarter → month)
Slice	Select one value of one dimension (e.g., Year = 2025)
Dice	Select a sub-cube by conditions on multiple dimensions
Pivot (Rotate)	Rotate axes to view data from another perspective

8.5 OLTP vs OLAP

Feature	OLTP (Operational DB)	OLAP / Data Warehouse
Purpose	Day-to-day transactions	Analysis & decision support
Data	Current, detailed	Historical, summarised

Feature	OLTP (Operational DB)	OLAP / Data Warehouse
Operations	Many short INSERT/UPDATE/DELETE	Complex read-heavy queries
Design	Normalised (ER)	Denormalised (star/snowflake)
Users	Clerks, applications	Analysts, managers
Size	MB – GB	GB – TB/PB

8.6 Data Mining

Data mining is the process of discovering **previously unknown, valid and useful patterns** from large datasets using statistics, machine learning, and database techniques. It is the core step of **KDD (Knowledge Discovery in Databases)**: Data cleaning → Integration → Selection → Transformation → **Mining** → Pattern evaluation → Knowledge presentation.

8.7 Data Mining Techniques

Technique	Purpose	Example / Algorithms
Association rule mining	Find items that occur together	Market-basket analysis: {bread} → {butter}; Apriori, FP-Growth
Classification	Assign data to predefined classes (supervised)	Spam detection; Decision tree, Naïve Bayes, SVM, k-NN
Clustering	Group similar objects without labels (unsupervised)	Customer segmentation; k-Means, DBSCAN, hierarchical
Regression / Prediction	Predict continuous values	Sales forecasting; linear regression
Anomaly / Outlier detection	Find unusual records	Fraud detection
Sequential pattern mining	Find patterns over time	Web click-streams, purchase sequences

Association rule measures: *Support* = P(A ∪ B); *Confidence* = P(B | A) = support(A ∪ B) / support(A); *Lift* = confidence / support(B).

8.8 Data Warehouse vs Data Mining

A data warehouse **stores and organises** clean integrated data; data mining **analyses** that data to extract knowledge. The warehouse is therefore the ideal source for data mining, and applications include retail analytics, banking fraud detection, healthcare, telecom churn prediction, and recommendation systems.

Conclusion

This document covered the theoretical foundations of query languages (tuple and domain relational calculus), the design goals of schema decomposition (dependency preservation), how the DBMS executes queries efficiently (query optimization), the protection of data (database security and the DAC, MAC and RBAC models), the manipulation of data and its integrity concerns (insertion and deletion), a major practical attack (SQL injection), and finally the analytics side of databases (data warehousing and data mining).

References

- Silberschatz, Korth & Sudarshan – *Database System Concepts*, McGraw-Hill.
- Elmasri & Navathe – *Fundamentals of Database Systems*, Pearson.
- Han, Kamber & Pei – *Data Mining: Concepts and Techniques*, Morgan Kaufmann.
- OWASP – SQL Injection Prevention Cheat Sheet (owasp.org).